

The Jenkinsfile orchestrates the following stages:

a. Checkout

- Pulls the latest code from the GitHub repository.
- Uses the main branch as the deployment source.

b. Build microservices

- A servicesToBuild map defines the build context for each microservice.
- For each entry:
 - Docker images are built using multistage Dockerfiles.
 - Maven is used to compile Spring Boot services.

c. Push Docker images

- Images are tagged and pushed to Docker Hub under the kanang namespace using credentials stored in Jenkins (DockerHubCred).

d. Deploy to Kubernetes via Ansible

- Uses an Ansible playbook (deploy-k8s.yml) to roll out the manifests to a live cluster.
- Kubernetes manifests are located in the *k8s/* directory and deployed to the *mindnotes* namespace.
- UTF-8 encoding is enforced to avoid issues during YAML parsing.

Environment variables: These values are set within the Jenkins pipeline for dynamic configuration:

Variable	Purpose
DOCKER_HUB_USER	Docker Hub account for image tagging
IMAGE_TAG	Docker image tag (set to latest)
K8S_NAMESPACE	Kubernetes namespace for deployment
K8S_MANIFEST_PATH	Location of YAML manifests inside repo

Multistage Dockerfiles significantly reduced image sizes, improving deploy speed. Automating Ansible execution inside Jenkins required us to preinstall the Kubernetes Ansible collection and properly set `ANSIBLE_COLLECTIONS_PATH`. Jenkinsfile modularisation helped separate concerns for build vs deployment. The complete project source code, along with all automation scripts, is available in our GitHub repository and can be accessed at https://github.com/Kanan-30/HealSphere_Project.

Containerisation and Kubernetes deployment

Each backend microservice in HealSphere is containerised using Docker, following a consistent multistage build process. By separating the build and runtime environments, we ensured smaller image sizes and faster deployment cycles. This also helped eliminate Maven and Node.js dependencies from the final production containers.

Dockerisation strategy: Each service (Auth, Appointment,

Group, Resource) has its own Dockerfile. While the base structure remains consistent, minor changes were made for services that include static file serving or need custom ports.

Kubernetes manifests and namespaces: Deployment to the cluster is handled using Kubernetes manifests stored in the *k8s/* directory in the project repository. Each microservice has its own set of YAMLs:

- Deployment.yaml
- Service.yaml
- ConfigMap.yaml (optional)
- Ingress.yaml (for API gateway or frontend)

All services are deployed into a dedicated namespace called *mindnotes*, which isolates HealSphere from other deployments and simplifies resource tracking.

Ansible-powered deployment: Instead of manually applying manifests, we automated deployments using an Ansible playbook (deploy-k8s.yml). This playbook:

- Iterates through service-specific manifest folders.
- Applies them to the target namespace.
- Supports rollbacks using Git-based tag versioning (planned for future use).

We found this method more scalable than basic kubectl apply, especially when rebuilding or resetting the cluster.

Ingress and service exposure: For external access:

- The frontend is exposed via an ingress controller (Traefik/ Nginx depending on environment).
- All services are internal (ClusterIP) and only reachable through the API gateway.
- The ingress rules are defined using host-based routing and TLS can be enabled via annotations if required.

Observations from production testing

- Applying resource limits (CPU, memory) was essential to prevent noisy-neighbour issues between services.
- Keeping readinessProbe and livenessProbe settings accurate prevented false restarts during service boot time.
- Running pods in separate namespaces with descriptive labels helped tremendously in log aggregation and dashboard filtering.

ELK Stack for monitoring

A reliable monitoring pipeline is essential for tracking application behaviour, diagnosing issues, and maintaining overall system health. For HealSphere, we implemented a logging and observability stack using ELK (Elasticsearch, Logstash, Kibana), with Filebeat acting as the log shipper within our Kubernetes environment.

Stack components and architecture

- **Filebeat (or Fluentd):** In Kubernetes, Filebeat is deployed as a DaemonSet, which ensures one instance runs on each node. It reads logs from container stdout and stderr streams—captured by Docker and exposed